

Rounds Protocol

User Stories & On-Chain Requirements

Capstone Assignment 2

PART A: User Stories & On-Chain Requirements Document

Section 1 — Core User Personas

Persona 1 — Circle Creator (automatically Position 1)

The circle creator is the user who initialises a new savings circle by setting all dynamic parameters — contribution amount, number of members, cycle duration, and premium percentage. By virtue of being the first to join, the creator is automatically assigned position 1 in the payout order. This makes the circle creator and position 1 member the same person — there is no separation between these roles at any point in the protocol.

This persona carries the highest collateral requirement of any participant — calculated dynamically as the total number of remaining cycles after their payout multiplied by the contribution amount — because they receive the pooled savings earliest and have the most remaining obligations. In exchange they receive a bonus payment from every other member who joins, compensating them for taking the highest-risk position.

Example for context: in a 10-member circle with a \$100 contribution amount and 10% premium, the circle creator locks \$900 as a security deposit and receives \$10 from each of the 9 other members who join — totalling \$90 in bonus payments.

Persona 2 — Circle Member, Standard Position (Positions 2 through N-1)

Standard position members are all participants who join after the creator and before the last available seat is filled. Their position in the payout order is assigned automatically based on the order in which they join — no selection or bidding is involved. Their collateral requirement is calculated dynamically from their assigned position — decreasing as position number increases.

In the first round these members pay the base contribution amount plus the configured premium, with the premium routed directly to the first member. From the second round onwards they pay only the base contribution amount. They receive the full pooled savings amount in their designated round and claim their security deposit back at circle completion.

Example for context: in a 10-member circle with a \$100 contribution amount, position 5 locks \$500 as a security deposit and pays \$110 in round 1.

Persona 3 — Circle Member, Final Position (Position N)

The final position member is the last person to fill the last available seat in a circle. Their position is assigned automatically by join order. Because they receive the pooled savings in the final round — after which they have no remaining obligations — their collateral requirement is always zero regardless of the circle's contribution amount or member count.

This persona pays the base contribution plus premium in round 1 and the base contribution only in all subsequent rounds. They receive the full pooled savings amount in the final round with absolutely nothing held back. They have no security deposit to claim at completion because nothing was locked at join time.

Example for context: in a 10-member circle with a \$100 contribution amount, position 10 locks \$0, pays \$110 in round 1, and receives the full \$1,000 pot in round 10 with every dollar freely spendable.

Persona 4 — Defaulting Member

The defaulting member is any participant who fails to make their contribution payment before the configured cycle deadline passes. This persona is not a distinct user type at join time — any active circle member can become a defaulting member if they miss a deadline.

When a deadline passes without payment, the protocol automatically processes the default — seizing the member's full remaining security deposit and using it to cover the missed payment and all future missed payments. The circle continues without interruption for all remaining members. The defaulting member is permanently excluded from further participation. Their net financial outcome is always zero or negative — the security deposit lost always equals or exceeds the payment obligations avoided — making default always economically irrational.

Persona 5 — Protocol Administrator

The protocol administrator is the developer and deployer of Rounds Protocol. This persona operates exclusively at the infrastructure layer — deploying the program, initialising global configuration, setting fee parameters, and holding emergency pause authority. The protocol administrator has no role or presence in the day-to-day operation of any savings circle and no interaction surface in the main frontend application. All administrator interactions happen via command line or a separate internal tool.

Section 2 — Function Map

Circle Creator — Position 1

- Start a new savings circle by submitting dynamic parameters
- Join the circle as the first member and lock security deposit
- Receive bonus payment from each subsequent member who joins
- Receive the full pooled savings amount in round 1
- Make regular savings payments every round from round 2 onwards
- Claim security deposit back at circle completion

Circle Member — Standard Position

- Browse available open circles and view their details
- Join an available circle and have position assigned automatically
- Make first round payment including base amount plus premium
- Make regular savings payments every round after the first
- Receive the full pooled savings amount in designated round
- Claim security deposit back at circle completion

Circle Member — Final Position

- Join the last available spot in a circle with zero security deposit requirement
- Make first round payment including base amount plus premium
- Make regular savings payments every round after the first
- Receive the full pooled savings amount in the final round with nothing held back

Defaulting Member

- Miss a payment deadline
- Have security deposit automatically seized to cover missed payment
- Be permanently excluded from further participation
- Experience net zero or negative financial outcome from defaulting

Protocol Administrator

- Deploy program and initialise global protocol configuration
- Set dynamic fee parameters at initialisation
- Update fee parameters post-launch if needed
- Pause all circle activity in an emergency
- Resume normal circle activity after emergency resolution

Section 3 — Final Refined User Stories

Circle Creator — Position 1

1. As a user I want to start a new savings circle by entering the contribution amount, number of members, cycle duration, and premium percentage so that the circle is created and visible for others to join
2. As the circle creator I want to join the circle I just created as the first member so that I am automatically given the first payout position and my security deposit is held by the protocol
3. As the first member I want to receive a bonus payment directly into my wallet each time another member joins the circle so that I am rewarded for joining first and locking the largest security deposit
4. As the first member I want to receive the full pooled savings amount in the first round so that I can access and spend every unit of it without any restrictions
5. As the first member I want to make my regular savings payment every round after I have received the pooled amount so that I complete my commitment to the other circle members

6. As the first member I want to get my security deposit back after the circle finishes so that the funds I locked upfront are returned to me in full

Circle Member — Standard Position

7. As a user looking to join a circle I want to see all available circles and their details — savings amount, number of members, round duration, and payout schedule — so that I can choose the circle that suits me
8. As a user I want to join an available circle so that I am automatically placed in the next open spot in the payout order and my security deposit is held by the protocol
9. As any member joining after the first member I want to make my first round payment — which includes my regular savings amount plus a small bonus for the first member — so that the bonus goes directly to the first member and my savings amount joins the shared pool
10. As any member I want to make my regular savings payment every round after the first so that the shared pool grows correctly and I stay in good standing with the circle
11. As a member I want to receive the full shared pool in my designated round so that I can access and spend every unit of it without any restrictions
12. As a member I want to get my security deposit back after the circle finishes so that the funds I locked when joining are returned to me in full

Circle Member — Final Position

13. As a user I want to join the last available spot in a circle so that I am placed in the final payout position and do not need to provide any security deposit
14. As the last member I want to receive the full shared pool in the final round with absolutely nothing held back or locked so that every unit of the pooled savings is mine to spend freely

Defaulting Member

15. As any member who misses a payment deadline I want the protocol to automatically use my security deposit to cover my missed payment the instant the deadline passes so that the circle continues without interruption and no other member is affected
16. As any member in a circle where another member has missed payments I want the circle to keep running exactly as normal so that my payout and security deposit return happen as scheduled without any disruption

Protocol Administrator

17. As the protocol administrator I want to set up the protocol with the correct fee settings before it goes live so that it operates correctly from the moment the first circle is created
18. As the protocol administrator I want to immediately stop all circle activity in an emergency so that every user's funds stay safely in place while the issue is being fixed
19. As the protocol administrator I want to resume normal circle activity after an emergency has been resolved so that users can continue participating without further disruption

Section 4 — On-Chain Requirements by User Story

Story 1 — Create a new savings circle

- `create_circle` instruction accepts contribution amount, total member count, cycle duration in slots, and premium basis points as dynamic inputs
- CircleAccount PDA derived from parameter fingerprint using seeds [`b"circle"`, `contribution_amount`, `total_members`, `cycle_duration_slots`, `premium_bps`, `token_mint`]
- If identical PDA exists in OPEN state, return error: "A circle with these settings is currently open and accepting members. Please join that circle instead"
- If identical PDA exists in READY, ACTIVE, COMPLETED, or CANCELLED state, allow new circle creation
- Check ProtocolConfig `is_paused` flag — revert if true
- Initialise CircleAccount storing all parameters, creator pubkey, state set to OPEN, member count set to zero, cycle set to zero

Story 2 — Join as first member and lock security deposit

- `join_circle` instruction assigns next position dynamically from current member count plus one
- Verify circle is in OPEN state
- Calculate collateral dynamically as remaining cycles after payout $\times$ contribution amount
- Transfer exact collateral from member's USDC token account to CollateralVault PDA
- Initialise MemberAccount PDA — seeds [`b"member"`, `circle_pubkey`, `member_pubkey`] — storing position, collateral locked, has received pot false, is defaulted false
- Initialise CollateralRecord PDA — seeds [`b"colrec"`, `circle_pubkey`, `member_pubkey`] — storing total locked, released zero, slashed zero
- Increment CircleAccount member count
- No premium transfer for position 1 — no prior member to receive it
- Check `is_paused` flag — revert if true

Story 3 — Receive bonus payment from each joining member

- `join_circle` instruction detects joining member is not position 1 when current member count is greater than zero
- Calculate premium dynamically as contribution amount $\times$ premium basis points / 10000
- Transfer premium from joining member's USDC token account directly to position 1's wallet in same transaction as collateral lock
- Retrieve position 1's wallet address from their stored MemberAccount PDA
- Premium transfer and collateral transfer are atomic — both succeed or both fail

Story 4 — Receive full pooled savings in round 1

- `disburse_pot` instruction verifies circle is ACTIVE
- Verifies current cycle number matches recipient's position
- Verifies all active non-defaulted members have paid via PaymentRecord bitmask
- Calculates protocol fee as pot total $\times$ fee basis points / 10000

- Transfers protocol fee from PotVault to fee recipient stored in ProtocolConfig
- Transfers full remaining pot to position 1's wallet — no deductions, no auto-locking
- Updates MemberAccount has received pot to true
- Increments cycle counter, sets next deadline slot
- Checks `is_paused` flag — reverts if true

Story 5 — Pay regular savings amount each round after round 1

- `pay_contribution` instruction verifies circle is ACTIVE
- Verifies current slot has not passed cycle deadline slot
- Verifies member is not defaulted
- Verifies member has not already paid this cycle via PaymentRecord bitmask
- Verifies payment amount equals exactly the contribution amount
- Transfers contribution amount from member's USDC token account to PotVault
- Sets member's bit in PaymentRecord bitmask for current cycle
- No premium involved from cycle 2 onwards
- Checks `is_paused` flag — reverts if true

Story 6 — Claim security deposit at circle completion

- `claim_collateral` instruction verifies circle is COMPLETED
- Verifies member has not already claimed via claimed flag in CollateralRecord
- Calculates claimable amount as total locked minus total slashed from CollateralRecord
- Transfers claimable amount from CollateralVault to member's USDC token account
- Updates CollateralRecord — total released set to claimable amount, claimed flag set to true
- Checks `is_paused` flag — reverts if true

Story 7 — Browse available circles

- No instruction required — read-only account fetch
- All circle details stored in CircleAccount PDA readable by any wallet
- Frontend queries using `getProgramAccounts` with memcmp filter on state field at consistent byte offset
- Payout schedule derivable from stored parameters — position N receives pot in cycle N

Story 8 — Join available circle with automatic position assignment

- Same `join_circle` instruction as Story 2
- Verifies circle is OPEN and not full
- Position assigned automatically — no user input required
- Collateral calculated automatically from position and contribution amount
- Reverts with descriptive error if member's USDC balance is insufficient
- Transitions circle from OPEN to READY when final seat fills

Story 9 — Make first round payment including premium

- `pay_contribution` instruction detects cycle 1 and member is not position 1
- Calculates premium as $\text{contribution amount} \times \text{premium basis points} / 10000$
- Splits payment into two atomic transfers — premium to position 1's wallet, base amount to PotVault
- Total deducted from member equals contribution amount plus premium amount

- Position 1's address retrieved from MemberAccount PDA
- Updates PaymentRecord bitmask after both transfers complete

Story 10 — Make regular savings payment every round after the first

- Same `pay_contribution` instruction for cycles 2 through N
- Transfers exactly contribution amount from member to PotVault
- Updates PaymentRecord bitmask
- No premium calculation or split transfer in cycles 2 through N

Story 11 — Receive full shared pool in designated round

- Same `disburse_pot` instruction for all positions in their respective cycles
- Verifies recipient's position matches current cycle number
- Transfers full pot remainder after fee to recipient's USDC token account
- PotVault balance reaches zero after each disbursement
- Cycle counter increments and next deadline slot is set atomically

Story 12 — Claim security deposit at completion

- Same `claim_collateral` instruction as Story 6
- Each member claims independently after circle reaches COMPLETED state
- No batch claim — every member calls individually

Story 13 — Join last seat with zero security deposit

- Same `join_circle` instruction
- Collateral calculation evaluates to zero for final position
- No token transfer to CollateralVault for zero collateral case
- MemberAccount still initialised with collateral locked set to zero
- CollateralRecord initialised with all amounts zero
- Circle transitions from OPEN to READY in this transaction
- `claim_collateral` handles zero collateral gracefully without reverting

Story 14 — Receive full shared pool in final round with nothing held back

- Same `disburse_pot` instruction for final cycle
- Verifies current cycle equals total members count
- Transfers full pot remainder after fee to final member's wallet
- Circle state transitions from ACTIVE to COMPLETED in same transaction
- All members' MemberAccounts verified as has received pot true or is defaulted true before COMPLETED transition

Story 15 — Security deposit automatically covers missed payment

- `process_default` instruction callable permissionlessly by any wallet
- Verifies circle is ACTIVE
- Verifies current slot is greater than cycle deadline slot
- Verifies identified member has not paid via PaymentRecord bitmask
- Verifies member is not already defaulted
- Transfers full remaining collateral from CollateralVault to PotVault
- Updates CollateralRecord — total slashed set to seized amount, total locked set to zero

- Updates MemberAccount — is defaulted set to true
- Updates PaymentRecord bitmask to mark defaulting member as covered
- Circle state remains ACTIVE — no state change
- Checks `is_paused` flag — reverts if true

Story 16 — Circle continues normally after a missed payment

- No new instruction — correctness requirement on `process_default` and `disburse_pot`
- `disburse_pot` treats defaulted member's covered slot identically to a paid slot
- PaymentRecord bitmask check excludes defaulted members from all-paid verification
- Remaining members' accounts completely unmodified by `process_default`
- Cycle deadline slot unchanged by default processing
- `disburse_pot` callable immediately after default processing if remaining members have paid

Story 17 — Set up protocol configuration before launch

- `initialize_protocol` instruction restricted to deployer keypair — callable once only
- Creates ProtocolConfig PDA with seeds `[b"config"]`
- Stores admin pubkey, fee basis points, fee recipient pubkey, `is_paused` initialised to false
- All parameters dynamic inputs — no hardcoded values
- Fails if ProtocolConfig PDA already exists
- `update_config` instruction restricted to admin pubkey — updates fee parameters post-launch
- Admin pubkey change requires separate two-step authority transfer to prevent accidental lockout

Story 18 — Pause all circle activity in emergency

- `pause_protocol` instruction restricted to admin pubkey
- Sets `is_paused` to true in ProtocolConfig atomically
- Every other instruction checks `is_paused` as first operation and reverts if true
- No funds moved or affected — all vault balances remain exactly in place
- Takes effect immediately on next instruction call after pause transaction confirms

Story 19 — Resume normal activity after emergency

- `unpause_protocol` instruction restricted to admin pubkey
- Sets `is_paused` to false in ProtocolConfig atomically
- All instructions resume normal execution immediately
- No circle state, member state, or vault balances modified
- `extend_deadline` instruction available to admin for post-emergency cycle deadline recovery if pause duration was significant

Section 5 — Complete On-Chain Requirements Summary

Accounts

Account	PDA Seeds	Stores
ProtocolConfig	[b"config"]	admin pubkey, fee bps, fee recipient, is_paused
CircleAccount	[b"circle", params fingerprint]	all circle parameters, state, cycle, deadline slot, member count
MemberAccount	[b"member", circle, member]	position, collateral locked, has received pot, is defaulted
CollateralRecord	[b"colrec", circle, member]	total locked, released, slashed, claimed flag
CollateralVault	[b"collateral_vault", circle]	USDC token account holding all circle collateral
PotVault	[b"pot_vault", circle]	USDC token account accumulating cycle contributions
PaymentRecord	[b"payment", circle, cycle]	u64 bitmask tracking paid members per cycle

Instructions

Instruction	Permission	Purpose
initialize_protocol	Admin only	Creates ProtocolConfig, sets fees
update_config	Admin only	Updates fee parameters
pause_protocol	Admin only	Sets is_paused true
unpause_protocol	Admin only	Sets is_paused false
extend_deadline	Admin only	Post-emergency deadline recovery
create_circle	Permissionless	Initialises CircleAccount from parameter fingerprint
join_circle	Permissionless	Assigns position, locks collateral, routes premium
start_circle	Permissionless	Transitions READY to ACTIVE, sets first deadline
pay_contribution	Any member	Accepts cycle payment, routes premium in cycle 1
disburse_pot	Permissionless	Transfers pot to cycle recipient after all paid
process_default	Permissionless	Seizes collateral, covers missed payment, marks defaulted
claim_collateral	Any member	Returns security deposit after COMPLETED
cancel_circle	Permissionless	Handles unfilled circles after timeout

State Machine

From	To	Triggered by
OPEN	READY	join_circle when final seat fills
READY	ACTIVE	start_circle once start conditions met
ACTIVE	ACTIVE	disburse_pot increments cycle counter
ACTIVE	COMPLETED	disburse_pot after final cycle
OPEN	CANCELLED	cancel_circle after fill timeout

Core Invariant At all times for every active non-defaulted member: collateral locked $\geq$ remaining cycles $\times$ contribution amount. Checked in join_circle, pay_contribution, and process_default. Any transaction violating the invariant reverts.

PART B: Process Appendix

Part A — Step 1: Manual User Brainstorming

Task

Based on the refined value proposition from Assignment 1, brainstorm every potential user type who might interact with Rounds Protocol across four categories — direct users, indirect users and beneficiaries, administrators and moderators, and stakeholders.

Output — Preliminary User List

Direct Users

- Circle creator — initialises new savings circle, automatically position 1
- Circle member early position — joins early, locks high collateral, receives pot early
- Circle member mid position — balanced risk and reward
- Circle member late position — locks less collateral, receives pot later
- Completing member — fulfilled all obligations, claiming returned collateral
- Defaulting member — missed contribution deadline, collateral seized

Indirect Users and Beneficiaries

- Non-participating observers — browsing circles considering joining
- Future circle participants — waiting for a circle to open
- DeFi ecosystem users — benefit from Rounds as a composable savings primitive

Administrators and Moderators

- Protocol deployer — puts program on Solana blockchain at launch
- Protocol admin — manages global configuration and emergency controls
- Permissionless crank caller — any wallet triggering automated instructions once conditions are met

Stakeholders

- Protocol fee recipient — receives fee from each pot disbursement
- Solana validator network — benefits from transaction volume
- Kamino Finance — post-MVP yield partner receiving TVL from collateral deposits
- Future protocol governance participants — post-MVP DAO or token holders

Part A — Step 2: AI-Assisted User Prioritization

AI Prompt Submitted

"My project's value proposition is: Rounds Protocol is a decentralized, trustless Rotating Savings and Credit Association (ROSCA) built on Solana — bringing one of the world's oldest and most universally practiced financial primitives onchain for the first time. A sliding-scale collateral mechanism mathematically eliminates default risk, making it impossible for any participant to profit from dishonesty without relying on reputation systems or trusted intermediaries. Early position

holders are compensated with a contribution premium funded collectively by later participants, creating balanced economic incentives across the full savings cycle. Here is my brainstormed list of all potential user types: Circle creator who is automatically position 1, Circle member standard position, Circle member final position, Defaulting member, Non-participating observer, Future circle participant, DeFi ecosystem user, Protocol deployer, Protocol admin, Permissionless crank caller, Protocol fee recipient, Solana validator network, Kamino Finance, Future protocol governance participants. Based on the value proposition, which 2-5 of these user types are the most critical to focus on for an initial Proof-of-Concept? For each user you recommend, provide a brief rationale explaining why they are essential for proving the project's core value."

AI Recommendations Summary

AI recommended circle creator and position 1 as separate users — corrected during manual analysis. Final AI recommendations after correction: Circle Creator who is automatically Position 1, Circle Member Standard Position, Circle Member Final Position, Defaulting Member, Protocol Admin.

Manual Analysis and Corrections

The AI incorrectly treated circle creator and position 1 as separate personas. This was corrected — whoever creates the circle is automatically position 1 by join order. No separation exists between these roles at any point. Additionally grace periods were removed from the protocol entirely — consistent with standard lending protocol behaviour. Default handling is fully automatic with no grace period. The permissionless default trigger behaviour is captured within the defaulting member user story rather than as a separate persona.

Final Prioritized User List

1. Circle Creator — automatically Position 1
2. Circle Member — Standard Position (Positions 2 through N-1)
3. Circle Member — Final Position (Position N)
4. Defaulting Member
5. Protocol Admin — supporting infrastructure user

Part A — Step 3: Core Function Mapping

AI Prompt Submitted

"For a project with this value proposition — Rounds Protocol is a decentralized, trustless ROSCA built on Solana where all circle parameters including contribution amount, member count, cycle duration, and premium percentage are dynamic and set at circle creation, position order is determined by join time, the circle creator is automatically position 1, a sliding scale collateral mechanism calculates each member's required collateral dynamically as remaining cycles after their payout cycle multiplied by the contribution amount, the final position always locks zero collateral — and focusing on these prioritized user types: Circle Creator who is automatically Position 1, Circle Member Standard Position, Circle Member Final Position, Defaulting Member, and Protocol Admin — help map out the key functions or interactions each user would need to perform."

Key Notes on Output

All parameters confirmed as dynamic throughout. Circle creator and position 1 confirmed as one person with no separation. Default handling confirmed as automatic with zero grace period. Protocol admin confirmed as entirely backend with no frontend presence.

Part A — Step 4: Deriving Core POC Requirements

Two Critical Interactions Identified

1. The full circle lifecycle — creation through completion including all contributions, all pot disbursements, and all collateral returns
2. The default and recovery flow — missed deadline, automatic collateral seizure, circle continuation uninterrupted

AI Prompt Submitted

"Rounds Protocol is a decentralized trustless ROSCA on Solana where all circle parameters are dynamic and set at circle creation. Position order is determined by join time. The circle creator is automatically position 1. A sliding scale collateral mechanism calculates each member's required collateral dynamically as remaining cycles after their payout multiplied by the contribution amount. The final position always locks zero collateral. A contribution premium is paid by all members except position 1 in the first cycle, routed directly to position 1. Based on these two critical user interactions: 1) The full circle lifecycle. 2) The default and recovery flow. What are the key technical requirements needed to build a proof of concept?"

Gap Identified

AI output did not explicitly include the duplicate circle prevention error message requirement. Added manually — if a circle with identical parameters already exists in OPEN state, the program returns a descriptive error directing the user to join the existing circle instead.

Part B — Adversarial Analysis and Granularity Check

AI Prompt Submitted

Full consolidated user functions and requirements from Part A submitted with the question: "Do these user stories and requirements truly hit the mark? Are the requirements granular enough to map to specific technical components? What is missing or unclear?"

Key Critique Points and Responses

Atomicity gaps identified and resolved: Create circle and join circle are two separate instructions — corrected and separated. Premium receipt and pot receipt happen at different times — separated in refined stories. Collateral claim is per-member independent — clarified.

Account structures explicitly named: ProtocolConfig, CircleAccount, MemberAccount, CollateralRecord, CollateralVault, PotVault, PaymentRecord — all named with explicit PDA seeds and stored data fields.

State machine fully enumerated: OPEN, READY, ACTIVE, COMPLETED, CANCELLED — all states named with explicit triggering conditions.

Premium timing decision made: Premium transferred at join time — when each subsequent member calls `join_circle`, the premium is calculated and transferred directly to position 1's wallet in the same transaction. This is cleaner and more transparent than deferring to cycle 1.

Fee deduction mechanism specified: Applied at `disburse_pot` time. Calculated as $\text{pot total} \times \text{fee basis points} / 10000$. Transferred to fee recipient before pot remainder transferred to cycle recipient.

Cancelled state added: If a circle does not fill within a configurable timeout window, any member who joined can call `cancel_circle` to retrieve their locked collateral.

Part C — Refinement Log

Story	Change Made	Rationale
1	Removed "initialise," "dynamic parameters," "onchain"	De-jargon
2	"Collateral locked into vault" → "security deposit held by protocol"	De-jargon
3	"Contribution premium" → "bonus payment"	De-jargon
4	"Pot" → "pooled savings amount," "cycle" → "round"	De-jargon
5	"Base contribution amount" → "regular savings payment"	De-jargon
6	"Claim locked collateral" → "get security deposit back"	De-jargon
7	"Parameters" → explicit list of what parameters are	Clarity
8	"Assigned next available position by join order" → "automatically placed in next open spot"	De-jargon
9	Explained premium components in plain language	Clarity
9 + 14	Merged — identical action for different user types	No overlap
10 + 15	Merged — identical action for all members	No overlap
13	"Position N with zero collateral requirement" → "final payout position with no security deposit"	De-jargon
15	"Seize remaining collateral" → "automatically use security deposit to cover missed payment"	De-jargon, clarity of purpose
16	"Default" → "missed payments" throughout	De-jargon
17	Combined deploy and initialise into single setup story	Atomicity — one admin action from their perspective
19	Added <code>extend_deadline</code> note for post-emergency recovery	Gap identified during refinement

This is the complete submission document. Every section of Part A is clean and final. Every step of the process is documented in Part B with prompts, outputs, analyses, corrections, and rationale throughout.

Ready to move to the next assignment whenever you are.